

扩大测评结论

测了哪些题、模型短板、harness 该往哪改

2026年8月31日

本轮不是调试集那十几题，而是 `eval/long_eval.yaml` 的扩大跑：模型 `apodex-1.1`，一把 key、禁止并发。结果目录 `results/long-eval/20260831-000637/`。

约束没变：不换模型、不换基准、不再跑 BrowseComp。下面只根据本轮分数谈 **harness 该改什么**。

一、测了哪些题

四套环境，共约 **144 个评测单元**。正赛接口是 `executable-world` 那一类：断网、typed actions、有限预算、隐藏真值、过程可打分。

套件	规模	测了什么	本轮结果
executable-world	5/5 全练习题	提交路径、核查门、去污、选列拟合、采购陷阱	均分约 0.98
TheAgentCompany	2 道安装题	容器内装 Go 1.17、装 OpenJDK 17	均为 2/2
ScienceAgentBench	verified 102 题	按任务说明写 Python；本地语法+短执行（非官方 Docker）	非空程序 59；语法 83；执行 0
DiscoveryWorld	35 题	8 个科学场景 × Easy/Normal/Challenge，加 10 个 Small Skills 与 Tutorial；每题最多 50 步	0 题通关

`executable-world` 分项：`verify_solutions` 1.0，`clinical_signal` 1.0，`corpus_dedup` 0.9885，`treatment_response` 1.0067，`corpus_procurement` 0.90。采购仍错判 decoy、漏掉蜜罐源，但已经能交可评分的包。

DiscoveryWorld 只有 Reactor Lab 蹭到过程分（Easy 2、Normal 2、Challenge 3，归一化约 8%–18%）。其余科学题与全部 Small Skills 均为完成=否、分=0。轨迹以 `WAIT` 和乱 `TELEPORT` 为主。

TheAgentCompany 的 GitLab / 回帖题本轮未跑（服务镜像不齐）。ScienceAgentBench 的 0 执行成功 **不能** 当官方成功率：本地缺 conda/Docker 评测栈，只能说明「交得出代码、在当前环境跑不赢」。

二、分数在说什么

同一套模型、同一把 key，短回合「选动作→看反馈→再提交」能得分；长回合「在世界里发现、再写能跑的科学程序」几乎不得分。这不是题太少，是能力切面不同。

已经证明通的。 typed action 协议、预算、提交提醒，在 `executable-world` 上够用。安装类 TAC 说明：把「怎样才算做完」写进 system prompt（例如 Go 必须 `ln -sf` 到 `/usr/bin`、禁止过早 `done`），弱模型也能跟着做完检查点。

正赛更像后一类。 TRACES 要的是隐藏真值上的发现过程，不是五道练习题的均分。DiscoveryWorld 的 0/35 比 EW 的 0.98 更接近正赛风险。

三、模型短板（按能力，不按榜名单点）

1. 不会做「有进度的探索」

Small Skills 是捡东西、开门、对话、给物品，不是解谜。35 题全灭，说明失败发生在 **感知→选合法动作→确认环境变了** 这一层，而不是科学推理太难。

当前 DW 环每步把任务条、动作表、传送点、上一步结果整段塞进 prompt，再要一个 JSON。模型的稳定策略是传送或空等。Stall 策略只接在 EW 上，DW 环根本没接；50 步里重复 WAIT 不会被打断。

2. 没有工作记忆，假设留不住

科学发现要：记下测过什么、仪器读数、对话选项、背包物品。现在每步几乎是无状态的。ContextPolicy 在 EW 里也是死的（context_fraction 从不更新），只是 EW 步数短、提交契约清楚，所以没爆。DW 50 步会把旧观察压过新线索。

3. 用「文本里写 JSON」当工具协议，弱模型很吃亏

EW / DW / TAC 都是「回复一个 JSON 对象」。解析失败就浪费一轮。DW 里 JSON 一坏就退化成 WAIT。Apodex 的 chat completions 兼容 tools，harness 没用上。

4. 科学代码是一次生成，没有「跑了再改」

SAB 每题一轮生成、max_tokens=4096。没有把 stderr、缺包、输出路径打回模型。59 条非空、本地执行 0，符合「会写不像会交可运行物」。正赛要的是可执行提交，不是看起来像程序的文本。

5. 收束策略按任务类型是反的

EW 在 40% 步数催 submit，对本轮练习题是加分项（必须交才有分）。同一套催促若套到 DW，会在还没做实验时乱结束；现在 DW 则是另一个极端——从不根据 scorecard 是否停滞而换策略。BrowseComp 时代「过早禁止再搜」是另一套复制出来的 loop。根因是一个 engine、多套互相抄的收束阈值。

6. 评测看见了结果，改完仍难对照过程

本轮有 summary.json 和心跳，这是进步。但 DW 没有「合法动作率 / 传送次数 / 对话次数 / scorecard 何时非零」这类过程计数；SAB 没有官方 Docker 分。改 harness 之后只能看通关数，看不出是解析好了还是真会探索。

四、Harness 往哪改

模型和题都不动。改的是 **策略层怎么接在 DW/SAB 上，以及 什么叫一次合法动作**。不要再堆 EW 题刷均分。

第一批：先让 Small Skills 不再全零

这是最便宜的探针。10 道原子技能若仍全零，后面科学场景不必重跑。

- DW 接上已有策略，不要另写一套散文 prompt。** Failure / Stall / Recovery 接到 dw_debug_agent。Stall 不要比原始观察字符串，要比：scorecard 是否增加、背包是否变化、是否连续 N 步 WAIT / TELEPORT。触发后注入可执行 hint：与视野内物体 USE、对 NPC 说话、读任务条里未完成项。
- 观察改成工作记忆，而不是整页 JSON。** 每步只给：任务未完成项、当前房间可见物体 uuid、合法动作短表、最近 1 次结果。另存「已传送地点 / 已对话选项 / 已测量值」，下一轮只给摘要。
- 动作协议改原生 tool_calls，JSON 文本只做兜底。** 解析失败不要默认 WAIT；重问一次，再失败再 WAIT。

4. **收束按任务类型配置**。EW：预算将尽才催 submit。DW：禁止用 submit 思维；步数后段催「对未完成任务做一次具体操作」。SAB：生成后必须进入执行-报错-修补，至少 1 轮，而不是生成即停。

第二批：让「提交物」真的可执行

5. **SAB 改成短闭环**。生成 → 语法检查 → 在 artifacts 目录执行（限时）→ 把 stderr 尾部和「有没有写出 output_fname」喂回模型再改。官方 Docker 分有 OpenAI 视觉裁判依赖，本仓库继续用本地执行当调试指标，不要绑 litellm。
6. **过程日志落到每题 jsonl**。DW：n_wait, n_teleport, n_parse_fail, score_delta。SAB：chars, syntax, ran, returncode。改 harness 后用 Small Skills × 10 和 SAB 前 10 条做 A/B，不要一改就重跑 35+102。

第三批：统一 loop，再碰办公栈

7. **一个 HarnessEngine，EW / DW / TAC 只换动作适配器**。现在 apodex_harness.solve 有策略，dw_debug_agent / tac_debug_agent / sab_debug_agent 没有。正赛只会有一套环境接口，调试时却在维护三套收束逻辑。
8. **ContextPolicy 真正压上下文**。每步更新 context_fraction，触发则保留 brief + 工作记忆 + 最近两次观察。
9. **TAC 办公题等 GitLab 镜像齐后再跑**。安装题满分不能外推到改 README / 回帖。那是另一套「检查点清单」prompt，不是本轮优先。

五、改完怎么验收

探针	现在	第一批算有效
EW 五题	均分 ~0.98	不回退（允许采购仍 <1）
DW Small Skills 10 题	0/10 通关	至少 3 题完成，或过程分明显非零
DW Reactor Lab	过程分 2-3	不下降即可
SAB 前 10 条	执行 0	至少出现语法+写出输出文件，不要求官方分

不要用 BrowseComp、不要加 worker、不要轮换 key。第一批只动 DW 环和动作协议；EW 高分当作回归锁，不当作成功标准。